

UPS 2025 Annual Shipping Performance

Converting the Power BI Template

From .pbit to .pbix — step by step, with verification

File: `dist/UPS_2025_Executive_Dashboard.pbit`

Contents: 8 tables · 11 relationships · 112 measures · 8 report pages · 70 visuals

Requires: Power BI Desktop (free), Windows. Roughly 15 minutes end to end.

Read this first

This template has not been opened in Power BI Desktop. It was generated in a Linux environment where no copy of Desktop exists. Everything checkable without one is validated — file encodings, the package manifest, the model schema, and every field reference in every visual — and the generator refuses to emit a file that fails those checks. But the first open is still a test. Section 10 covers what to do if it fails, and there are two working fallbacks.

1. Why there is a template and not a .pbix

A .pbix stores its data model in a part called DataModel: the Analysis Services tabular model, serialised in a proprietary compressed format that only Power BI Desktop and the Analysis Services engine can write. No external tool produces one — not Microsoft's, not anyone's.

A .pbix is the same package with the data removed and the model expressed as plain TMSL JSON, which can be authored externally. Opening it in Desktop loads your data and materialises a live model. The first Save As is the step that mints the .pbix.

So the template is not a lesser deliverable. It is the only form this can take before it has been through Desktop once, and **that pass is what these directions walk through.**

2. Before you start

You need	Detail
Power BI Desktop	Free from the Microsoft Store or powerbi.microsoft.com/desktop . Windows only — there is no Mac build.
The template	<code>dist/UPS_2025_Executive_Dashboard.pbix</code>
The repository	For the theme file and the DAX/M sources, in case you need a fallback.
Python 3.9+	Only to generate the data files in Step 1. Any recent version.
Your data	Either the real UPS extracts, or the synthetic sample that ships with the repo.

A note on the data path

The model reads CSV files from a folder you nominate. Power BI needs an **absolute** path — `C:\Users\you\ups-2025-executive-dashboard\data\processed`, not a relative one. Have that path ready before Step 3, with no trailing backslash.

3. Step 1 — Produce the data files

The model loads the star-schema CSVs written by the ETL, not the raw Excel extracts. Generate them first; the template will fail on load if the folder is empty.

Option A — your real UPS extracts

Place the monthly extracts in `data/raw/`, then run:

```
pip install -r etl/requirements.txt
python etl/ups_etl.py --raw data/raw --out data/processed --report
```

Note the absolute path of `data/processed`. Check the reconciliation summary it prints — if January's volume is exactly double what the January extract shows, the year-to-date deduplication did not run.

Option B — the synthetic sample

Use this to test the template, or to demonstrate the dashboard without exposing client data.

```
python etl/make_sample_data.py --out data/sample
```

What you should have

Seven CSV files plus a manifest in the output folder:

Fact tables	Dimensions and manifest
Fact_VolumeSpend.csv	Dim_Date.csv
Fact_TimeInTransit.csv	Dim_Account.csv
Fact_Claims.csv	Dim_ServiceMapping.csv
Fact_Accessorial.csv	manifest.json

If any are missing, the template fails with a file-not-found naming that exact file, which makes the cause obvious.

4. Step 2 — Open the template

Double-click `UPS_2025_Executive_Dashboard.pbit`, or from Desktop use File → Import → Power BI template.

Power BI Desktop opens a parameter dialog before it loads anything. This is normal and is the defining behaviour of a template — it is asking where your data lives.

5. Step 3 — Enter the parameters

Parameter	What to enter
p_DataFolder	The absolute path to the folder from Step 1. No trailing backslash. <code>C:\Users\you\ups-2025-executive-dashboard\data\processed</code>
p_ReportingYear	2025

Click Load.

The most common mistake

A trailing backslash, or a path to the repository root rather than to the folder containing the CSVs. Both produce a file-not-found error naming a path with a doubled separator or a missing folder level — read the path in the error message and it tells you which.

6. Step 4 — Let the model load

Desktop runs eight queries and builds the model. On the sample data this takes a few seconds; on a full year of real extracts, under a minute.

When it finishes you should see, in the Data pane on the right:

Item	Expected
Tables	8 data tables plus Volume Threshold, Share Threshold, and _Measures
_Measures	112 measures in five numbered display folders
Dim_ServiceMapping	Present but hidden — it is the two-vocabulary mapping table, needed by the Data Quality page and deliberately kept out of the field list
Report pages	8 tabs along the bottom, from Executive Overview to Data Quality

7. Step 5 — Two manual settings

Two things cannot be set reliably from the template and must be applied by hand. Both take under a minute and both matter.

5a. Apply the report theme

View → Themes → Browse for themes → select:

```
powerbi\theme\UPS_Executive_Theme.json
```

The theme carries a colour palette validated for colour-vision deficiency, hairline gridlines, data labels off by default, and no dual-axis configuration.

It is applied here rather than embedded in the template on purpose: a malformed custom-theme resource is one of the few things that can stop a .pbit opening at all, and that is not worth risking the whole file for one click.

5b. Set the month sort order

In the Data pane, expand Dim_Date and do this twice:

Action	
1	Select MonthName
2	Column tools ribbon → Sort by column
3	Choose SortOrder
4	Repeat all three for MonthYearLabel

Why this is not optional

Without it, every month axis in the report sorts alphabetically: Apr, Aug, Dec, Feb, Jan... Trend lines become meaningless and it is easy not to notice, because the chart still looks like a chart. Check the volume-by-month column chart on page 3 reads Jan through Dec before you go further.

8. Step 6 — Verify before you save

Six checks. Each catches a specific, known failure mode, and each takes seconds.

#	Check	What it catches
1	Page 3 volume chart reads Jan → Dec	The sort-by columns from Step 5b were not applied.
2	On-time delivery equals the <i>Grand Total</i> row of the source Time-in-Transit extract (96.46% for March 2025)	A reading around 98.2% means a percentage is being averaged rather than weighted — a 4-package service counted equally with a 48,000-package one.
3	Page 8 shows Reconciliation Status = Reconciled	The accessorial detail must sum to net spend to the cent. A variance means one source file is stale or partly loaded.
4	January volume matches the January extract, not double it	The year-to-date deduplication. February's extract contains January as well; a naive load double-counts it with no error.
5	Service Volume Mix % sums to 100% across services	If it shows 100% on every row, a measure is filtering a table with no relationship to the facts and the result is silently wrong.
6	The banner card on every page describes your actual coverage	With one month of transit data it must read PROVISIONAL, not ANNUAL.

If check 2 or 4 fails

Stop and fix it before saving. Both failure modes produce numbers that are internally consistent and completely wrong — nothing downstream will look odd, and the error will survive review. **Getting exactly double is the signature of the year-to-date overlap.**

9. Step 7 — Save as .pbix

Action	
1	File → Save as
2	Set Save as type to Power BI files (*.pbix)
3	Name it, for example <code>UPS_2025_Executive_Dashboard.pbix</code>
4	Save

That file is a full .pbix: model, data, and report in one. From here it behaves like any other — Publish to send it to the Power BI Service, or share the file directly.

Keep the template

The .pbit stays useful. It is a fraction of the size, contains no data, and is the thing to hand to a colleague who needs the same dashboard against their own folder. Regenerate it any time from the DAX sources with `python powerbi/build_pbit.py`.

10. Troubleshooting

The template will not open at all

Power BI Desktop is strict about package structure and its expectations shift between versions. If Desktop rejects the file, the error text usually names the part it did not like — note it, because it points straight at the fix.

Two working fallbacks, in order:

Fallback A — the Power BI Project

Open `dist\pbip\UPS_2025_Executive_Dashboard.pbip` instead.

This is Microsoft's supported text-based project format and uses a different, more tolerant loader than the .pbit package reader. It contains the same model and report.

You may need to enable it first: File → Options and settings → Options → Preview features → **Power BI Project (.pbip) save option**, then restart Desktop.

Fallback B — build it by hand

Slower, but everything needed is in the repository and nothing is lost:

Source	What it gives you
<code>powerbi/powerquery/*.m</code>	The queries, which read the raw extracts directly
<code>powerbi/dax/*.dax</code>	All 112 measures, in five files, ready to paste
<code>powerbi/model/relationships.md</code>	The relationship table and the reasoning behind three specific joins
<code>powerbi/report_spec/</code>	Page layouts, the slicer matrix, and interaction rules
<code>docs/06_build_checklist.md</code>	The build order, with a verification step for every item

Common errors during load

Symptom	Cause	Fix
"We couldn't find the file" naming a CSV	Wrong p_DataFolder, or Step 1 not run	Check the path is absolute and has no trailing backslash
A column type conversion error	The CSVs were opened and re-saved in Excel, changing date or number formats	Re-run the ETL; do not open the CSVs in Excel between steps
Blank visuals on every page, no error	The model loaded but the tables are empty	Check the CSVs have rows beyond the header line
Blank on-time delivery, everything else fine	No Time-in-Transit data for the months loaded	Expected if only some months have transit extracts — the banner will say so
Month axes alphabetical	Step 5b not applied	Set the sort-by columns

11. After the .pbix exists

The template deliberately omits several things that are faster to add in Desktop than to hand-author into the report definition. All are specified in `powerbi/report_spec/`.

Addition	Why it matters
Global synced slicers — Year, Quarter, Month, Service Group, Domestic/International, Account, State	The spec says which pages each appears on, and which pages deliberately omit one

Addition	Why it matters
Drillthrough pages — Service, Account, Month, Location detail	Keeps the main pages clean while allowing investigation. Account Detail is the workhorse, since claims are account-grain
Conditional formatting — data bars on the loss-rate index; banner background driven by [Data Basis Banner Colour]	Turns the concentration table from a list into an obvious answer
Bookmarks, including “Threshold: none”	Toggling it in a review shows a tiny service jumping to the top of the ranking, which makes the case for volume governance better than any slide
Set KPI cards to no interaction	The default is Highlight; a user who clicks a card and sees the page change assumes the click meant something
Error bars on the on-time chart from [OTD Wilson Lower Bound]	Shows how much of each percentage is knowledge and how much is a small sample

12. Moving to direct refresh from the raw extracts

The template loads CSVs produced by the Python ETL. That was a deliberate choice: the Power Query implementation that reads the raw UPS folder directly is several hundred lines of M that has never been executed, and a template that fails on load is worse than none. The ETL has been run against the real extracts and reconciles to them exactly.

Once the model is open and you can iterate in Desktop, switching is contained work:

Step	
1	Add the parameters from <code>powerbi/powerquery/00_Parameters.m</code>
2	Add the helper functions from <code>01_fnHelpers.m</code> as blank queries, named exactly as shown
3	Replace one fact query at a time, starting with Volume & Spend
4	After the first, reconcile January against the source extract. This is where the year-to-date double-count would appear.
5	Repeat for Time-in-Transit, Claims and Accessorial

Why January is the test

The February extract is a year-to-date file: it contains January as well as February. The January extract contains January. A conventional Get Data → Folder append — the obvious approach — double-counts January exactly. Measured on the real files that is a 100% overstatement of the month, with no error, no warning, and a total that still looks entirely plausible.

Reference: [docs/08_opening_the_powerbi_file.md](#) in the repository carries the same steps in short form, plus notes on regenerating the template.